

# Git과 GitHub을 활용한 버전 관리와 협업

R 프로그래밍 with workflow design



# Git과 GitHub의 이해

- **Git**: 코드 변경 이력을 추적하고 복원할 수 있는 버전 관리 시스템
- **GitHub**: Git 기반 원격 저장소 플랫폼
- 주요 장점
  - 변경 사항 자동 추적
  - 실험용 브랜치 생성
  - 협업 및 병합(merge) 지원

# Git 설치 및 사용자 설정

- Git 설치: Git 공식 웹사이트
- 커밋 작성자 정보를 위해 Git 사용자 정보 등록 필요
- RStudio에서 Terminal 탭 열기 (`Tools > Terminal > New Terminal`)
- 사용자 이름과 이메일 등록:

```
git config --global user.name "홍길동"
git config --global user.email "hong@example.com"
```

- 설정 확인:

```
git config --global --list
```

# GitHub 계정 및 SSH 키 설정

- GitHub 계정 생성
  - Git 공식 웹사이트에서 무료 가입
  - 이메일은 Git 프로필 설정 이메일과 일치하는 것이 좋음
- SSH 키 설정 (비밀번호 없이 인증): `RStudio > Tools > Global Options > Git/SVN`
  - Create SSH Key 클릭해 키 생성
  - View public key로 공개 키 확인
- GitHub에 로그인 후 `Settings > SSH and GPG keys > New SSH Key`에 RStudio에서 생성한 공개 키 붙여넣기

# Git 연동된 R 프로젝트 생성 방법

- `RStudio > File > New Project`
- 선택 옵션
  - New Directory: 새 폴더 생성 후 `Create a git repository` 체크하여 Git 초기화
  - Existing Directory: 기존 폴더에 `.Rproj` 추가 + Git 리포지토리로 설정
  - Version Control: GitHub URL 입력 후 레포지토리 클론 (`.git`, `.Rproj`, 프로젝트 구조 등 자동 구성)
- 선택적 설정 옵션:
  - renv 활성화 (패키지 환경도 Git과 함께 관리)
  - `.gitignore` 자동 생성됨 (예: `.Rhistory`, `.RData` 등 제외)

# GitHub 레포지토리 클론

- `RStudio > File > New Project > Version Control`
- 클론(clone): GitHub의 저장소를 로컬로 복제하여 동일한 Git 프로젝트 환경을 구성하는 것
- Repository URL에 GitHub 레포지토리 주소 입력하면 로컬 프로젝트로 클론됨
- `.Rproj`, `.git` 포함된 Git 연동 프로젝트 자동 생성

# Git의 핵심 개념

- 스테이징: 커밋할 변경 파일 선택
- 커밋: 변경 이력 저장 (git commit -m “메시지”)
- 브랜치: 독립적 작업 공간 생성 및 병합
- 리셋/리버트: 이전 상태로 되돌리기

# GitHub 연동과 푸시

- 레포지토리 연결:

```
git remote add origin "REPO address"
git branch-M main
git push-u origin main
```

- Pull/Pull Request를 통해 협업

# GitHub 협업 워크플로

- `GitHub > Settings > Collaborators` 에서 팀원 추가
- Issues / Projects / Branch 활용
- Pull Request(PR)로 변경 제안 후 병합
- 충돌 발생 시 수정 후 커밋